

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
from scipy.stats import zscore
```

Question #1: In this exercise, you are provided with a dataset of employees from different departments in a company. The dataset contains the following columns: (25 points) • EmployeeID, Department, Salary, Experience, Age, Education, PerformanceScore

a) Load the CSV file eda_employee_data.csv into a pandas DataFrame.

```
df = pd.read_csv('/content/eda_employee_data-1.csv')
print(df)
```

	EmployeeID	Department	Salary	Experience	Age	Education	PerformanceScore
0	201	Sales	58000		25	Bachelors	3.2
1	202	HR	62000		29	Masters	3.5
2	203	IT	75000		35	Bachelors	4.0
3	204	IT	78000		40	PhD	4.2
4	205	HR	61000		31	Masters	3.6
5	206	Sales	57000		24	Bachelors	3.1
6	207	IT	150000	15	55	PhD	4.9
7	208	Marketing	67000		30	Masters	3.8
8	209	Finance	72000		38	PhD	4.1
9	210	Sales	59000		26	Bachelors	3.3

b) Use describe() to compute summary statistics for numeric columns.

```
# Print summary statistics
print(df.describe())
```

	EmployeeID	Salary	Experience	Age	PerformanceScore
count	10.00000	10.000000	10.00000	10.000000	10.000000
mean	205.50000	73900.000000	4.90000	33.300000	3.770000
std	3.02765	27746.671472	3.95671	9.333929	0.549848
min	201.00000	57000.000000	2.00000	24.000000	3.100000
25%	203.25000	59500.000000	2.25000	26.750000	3.350000
50%	205.50000	64500.000000	3.50000	30.500000	3.700000
75%	207.75000	74250.000000	5.75000	37.250000	4.075000
max	210.00000	150000.000000	15.00000	55.000000	4.900000

c) Using Tukey's Fences, identify outliers in the Salary and Experience columns.

```
# Using Tukey's Fences, identify outliers in the Salary and Experience columns.
```

```
def find_outliers(df, column):
    Q1 = df[column].quantile(0.25)
    Q3 = df[column].quantile(0.75)
    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    outliers = df[(df[column] < lower_bound) | (df[column] > upper_bound)]
    return outliers
```

```
salary_outliers = find_outliers(df, 'Salary')
experience_outliers = find_outliers(df, 'Experience')
```

```
print("Outliers in Salary:\n", salary_outliers['Salary'])
print("\nOutliers in Experience:\n", experience_outliers['Experience'])
```

```
Outliers in Salary:
6    150000
Name: Salary, dtype: int64

Outliers in Experience:
6    15
```

Name: Experience, dtype: int64

d) Use z-score method to detect outliers in the Age column ($|z| > 3$).

```
# Use z-score method to detect outliers in the Age column ( $|z| > 3$ )
z_scores = zscore(df['Age'])
age_outliers = df[np.abs(z_scores) > 3]

if age_outliers.empty:
    print("No outliers detected in Age.")
else:
    print("Outliers in Age:", " ".join(map(str, age_outliers.tolist())))
```

➡ No outliers detected in Age.

e) Print all rows that were identified as outliers in either method, with a note of which column they were outliers in

```
# Print all rows that were identified as outliers in either method, with a note of which column they were outliers in
outliers = pd.concat([salary_outliers.assign(Outlier_Column="Salary"),
                      experience_outliers.assign(Outlier_Column="Experience"),
                      age_outliers.assign(Outlier_Column="Age")])

if age_outliers.empty:
    no_age_outliers = pd.DataFrame([{"EmployeeID": "N/A",
                                     "Department": "N/A",
                                     "Salary": "N/A",
                                     "Experience": "N/A",
                                     "Age": "No Age Outliers Detected",
                                     "Education": "N/A",
                                     "PerformanceScore": "N/A",
                                     "Outlier_Column": "Age"}])
    outliers = pd.concat([outliers, no_age_outliers], ignore_index=True)
else:
    outliers = pd.concat([outliers, age_outliers.assign(Outlier_Column="Age")])

# if not age_outliers.empty:
#     outliers = pd.concat([outliers, age_outliers.assign(Outlier_Column="Age")])

outliers = outliers.drop(columns=["Age_zscore"], errors="ignore")

print(outliers)
```

➡

	EmployeeID	Department	Salary	Experience	Age	
0	207	IT	150000	15	55	
1	207	IT	150000	15	55	
2	N/A	N/A	N/A	N/A	No Age Outliers Detected	

	Education	PerformanceScore	Outlier_Column
0	PhD	4.9	Salary
1	PhD	4.9	Experience
2	N/A	N/A	Age

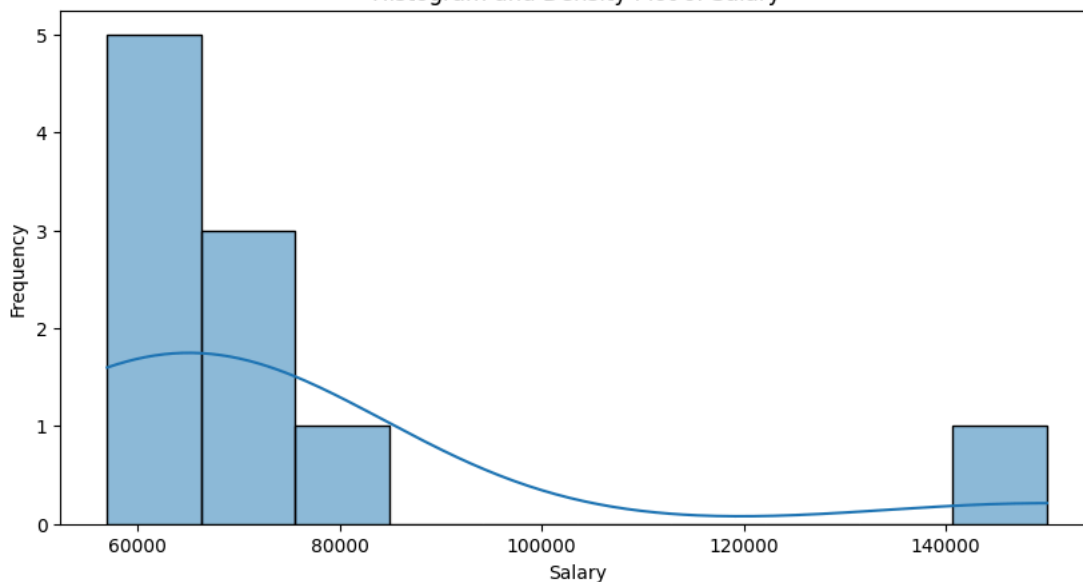
Question #2:

a) Plot a histogram and a density plot of the Salary column using `sns.histplot()` and `sns.kdeplot()`

```
#Plot a histogram and a density plot of the Salary column using sns.histplot() and sns.kdeplot()
plt.figure(figsize=(10, 5))
sns.histplot(df["Salary"], bins=10, kde=True)
plt.title("Histogram and Density Plot of Salary")
plt.xlabel("Salary")
plt.ylabel("Frequency")
plt.show()
```



Histogram and Density Plot of Salary

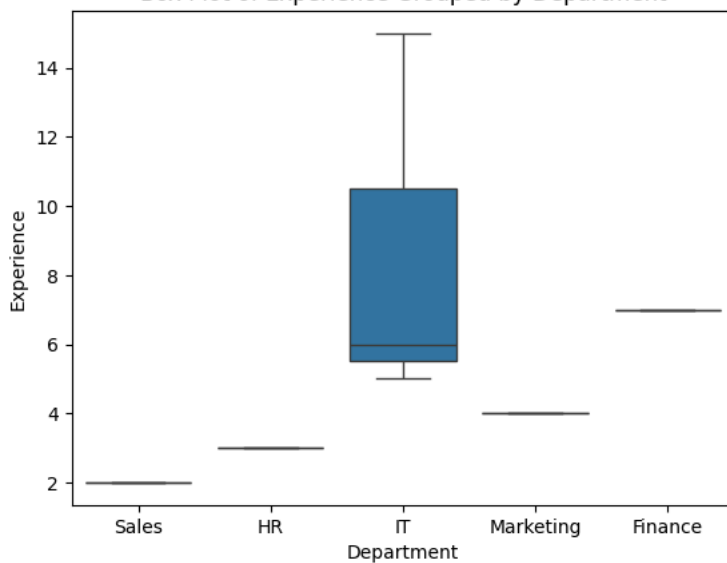


b) Create a box plot of Experience grouped by Department

```
#Create a box plot of Experience grouped by Department
boxplot = sns.boxplot(x='Department', y='Experience', data=df)
plt.title('Box Plot of Experience Grouped by Department')
plt.xlabel('Department')
plt.ylabel('Experience')
plt.show()
```

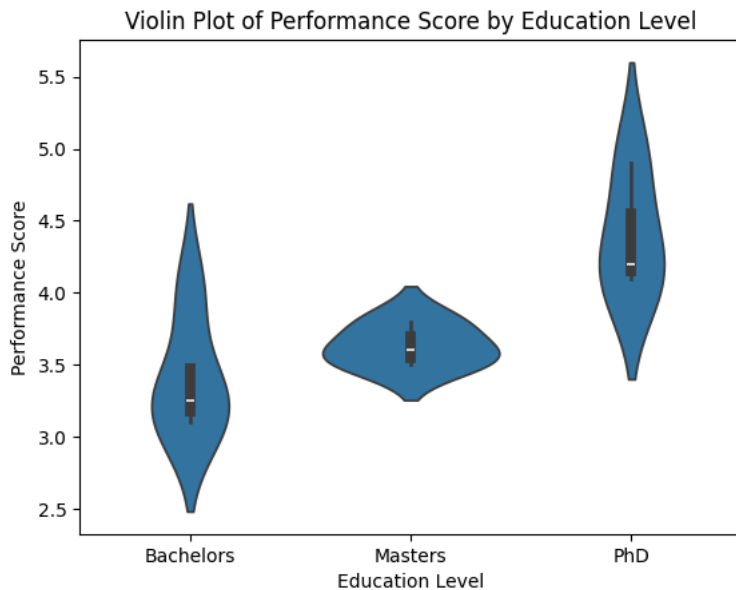


Box Plot of Experience Grouped by Department



c) Create a violin plot to visualize the distribution of PerformanceScore for each Education level.

```
#Create a violin plot to visualize the distribution of PerformanceScore for each Education level
violinplot = sns.violinplot(x='Education', y='PerformanceScore', data=df)
plt.title('Violin Plot of Performance Score by Education Level')
plt.xlabel('Education Level')
plt.ylabel('Performance Score')
plt.show()
```



d) Which visualization type made it easiest to identify skewness or outliers? Briefly explain

```
print("The violin plot made it easiest to identify skewness or outliers because\nit combines the information from a box plot and a kernel de
```



The violin plot made it easiest to identify skewness or outliers because it combines the information from a box plot and a kernel density plot, allowing for a comprehensive view of the distribution of data points within each category.

Question #3

You're given raw grade data with inconsistencies:

```
df2 = pd.DataFrame({
    "StudentID": [1001, 1002, 1003, 1004, 1005],
    "Name": ["Alice", "Bob", "Charlie", "D@vid", "Eve"],
    "Math": [88, 76, None, 91, 60],
    "English": [90, 85, 79, None, 73],
    "Science": [85, 78, 80, 82, None],
    "GradeLevel": ["10th", "10th", "11th", "11th", "10th"]
})

print(df2)
```



	StudentID	Name	Math	English	Science	GradeLevel
0	1001	Alice	88.0	90.0	85.0	10th
1	1002	Bob	76.0	85.0	78.0	10th
2	1003	Charlie	NaN	79.0	80.0	11th
3	1004	D@vid	91.0	NaN	82.0	11th
4	1005	Eve	60.0	73.0	NaN	10th

a) Identify and fix the incorrect name entry in the dataset using regex or string replacement

```
# Identify and fix the incorrect name entry in the dataset using regex or string replacement
df2['Name'] = df2['Name'].str.replace('@', 'a')
print(df2)
```



	StudentID	Name	Math	English	Science	GradeLevel
0	1001	Alice	88.0	90.0	85.0	10th
1	1002	Bob	76.0	85.0	78.0	10th
2	1003	Charlie	NaN	79.0	80.0	11th
3	1004	David	91.0	NaN	82.0	11th
4	1005	Eve	60.0	73.0	NaN	10th

b) Fill missing subject scores with the median of that subject for the same GradeLevel using groupby().transform()

```
# Fill missing subject scores with the median of that subject for the same GradeLevel using groupby().transform()
subjects = ['Math', 'English', 'Science']
df2[subjects] = df2.groupby('GradeLevel')[subjects].transform(lambda x: x.fillna(x.median()))
print(df2)
```

	StudentID	Name	Math	English	Science	GradeLevel
0	1001	Alice	88.0	90.0	85.0	10th
1	1002	Bob	76.0	85.0	78.0	10th
2	1003	Charlie	91.0	79.0	80.0	11th
3	1004	David	91.0	79.0	82.0	11th
4	1005	Eve	60.0	73.0	81.5	10th

c) Create a new column "WeightedScore" assuming weights: Math=40%, English=30%, Science=30%

```
# Create a new column "WeightedScore" assuming weights: Math=40%, English=30%, Science=30%.
df2['WeightedScore'] = df2['Math'] * 0.4 + df2['English'] * 0.3 + df2['Science'] * 0.3
print(df2)
```

	StudentID	Name	Math	English	Science	GradeLevel	WeightedScore
0	1001	Alice	88.0	90.0	85.0	10th	87.70
1	1002	Bob	76.0	85.0	78.0	10th	79.30
2	1003	Charlie	91.0	79.0	80.0	11th	84.10
3	1004	David	91.0	79.0	82.0	11th	84.70
4	1005	Eve	60.0	73.0	81.5	10th	70.35

d) Use pd.cut() to categorize performance into "Low", "Medium", "High" based on score distribution.

```
Generated code may be subject to a license | www.analyticsvidhya.com/blog/2022/02/auto-sklearn-accelerate-your-machine-learning-models-with-automl/
bins = [0, 75, 85, 100]
performance_levels = ['Low', 'Medium', 'High']
df2['PerformanceCategory'] = pd.cut(df2['WeightedScore'], bins = bins, labels=performance_levels)
print(df2)
```

	StudentID	Name	Math	English	Science	GradeLevel	WeightedScore	\
0	1001	Alice	88.0	90.0	85.0	10th	87.70	
1	1002	Bob	76.0	85.0	78.0	10th	79.30	
2	1003	Charlie	91.0	79.0	80.0	11th	84.10	
3	1004	David	91.0	79.0	82.0	11th	84.70	
4	1005	Eve	60.0	73.0	81.5	10th	70.35	

	PerformanceCategory
0	High
1	Medium
2	Medium
3	Medium
4	Low

e) Calculate class-wise average and standard deviation and flag students performing below one standard deviation

```
# Calculate class-wise average and standard deviation and flag students performing below one standard deviation
class_avg = df2.groupby('GradeLevel')['WeightedScore'].mean()
class_std = df2.groupby('GradeLevel')['WeightedScore'].std()

df2['Below_1_StdDev'] = df2.apply(lambda x: x['WeightedScore'] < (class_avg[x['GradeLevel']] - class_std[x['GradeLevel']]), axis=1)
print(df2)
```

	StudentID	Name	Math	English	Science	GradeLevel	WeightedScore	\
0	1001	Alice	88.0	90.0	85.0	10th	87.70	
1	1002	Bob	76.0	85.0	78.0	10th	79.30	
2	1003	Charlie	91.0	79.0	80.0	11th	84.10	
3	1004	David	91.0	79.0	82.0	11th	84.70	
4	1005	Eve	60.0	73.0	81.5	10th	70.35	

	PerformanceCategory	Below_1_StdDev
0	High	False
1	Medium	False
2	Medium	False
3	Medium	False
4	Low	True

Question #4

```
data = {
    "Date": ["2023-01-02", "2023-01-04", "2023-01-06", "2023-01-10", "2023-01-13"],
    "AAPL_Price": [150.0, 152.5, None, 155.0, 157.5],
    "MSFT_Price": [240.0, None, 245.0, 250.0, None],
    "Volume_Million": [5.1, 5.3, 5.0, None, 5.6]
}

df_trading = pd.DataFrame(data)
print(df_trading)
```



	Date	AAPL_Price	MSFT_Price	Volume_Million
0	2023-01-02	150.0	240.0	5.1
1	2023-01-04	152.5	NaN	5.3
2	2023-01-06	NaN	245.0	5.0
3	2023-01-10	155.0	250.0	NaN
4	2023-01-13	157.5	NaN	5.6

a) Convert the "Date" column to datetime, set it as the index, and ensure the index is sorted.

```
# Convert the "Date" column to datetime, set it as the index, and ensure the index is sorted
df_trading['Date'] = pd.to_datetime(df_trading['Date'])
df_trading.set_index('Date', inplace=True)
df_trading.sort_index(inplace=True)
print(df_trading)
```



	Date	AAPL_Price	MSFT_Price	Volume_Million
	2023-01-02	150.0	240.0	5.1
	2023-01-04	152.5	NaN	5.3
	2023-01-06	NaN	245.0	5.0
	2023-01-10	155.0	250.0	NaN
	2023-01-13	157.5	NaN	5.6

b) Reindex the DataFrame to include all business days from "2023-01-02" to "2023-01-13" using `pd.date_range()` with frequency "B". • Use `method='ffill'` and `limit=1` for the price columns • Use `fill_value=0` for the Volume_Million column

```
# Reindex the DataFrame to include all business days from "2023-01-02" to "2023-01-13" using pd.date_range() with frequency "B".
# Use method='ffill' and limit=1 for the price columns
# Use fill_value=0 for the Volume_Million column
business_days = pd.date_range(start='2023-01-02', end='2023-01-13', freq='B')
df_trading = df_trading.reindex(business_days)
df_trading[['AAPL_Price', 'MSFT_Price']] = df_trading[['AAPL_Price', 'MSFT_Price']].fillna(method='ffill', limit=1)
df_trading['Volume_Million'] = df_trading['Volume_Million'].fillna(0)
print(df_trading)
```



	Date	AAPL_Price	MSFT_Price	Volume_Million
0	2023-01-02	150.0	240.0	5.1
1	2023-01-03	150.0	240.0	0.0
2	2023-01-04	152.5	NaN	5.3
3	2023-01-05	152.5	NaN	0.0
4	2023-01-06	NaN	245.0	5.0
5	2023-01-09	NaN	245.0	0.0
6	2023-01-10	155.0	250.0	0.0
7	2023-01-11	155.0	250.0	0.0
8	2023-01-12	NaN	NaN	0.0
9	2023-01-13	157.5	NaN	5.6

<ipython-input-159-256897598aa5>:6: FutureWarning: DataFrame.fillna with 'method' is deprecated and will raise in a future version. Use
df_trading[['AAPL_Price', 'MSFT_Price']] = df_trading[['AAPL_Price', 'MSFT_Price']].fillna(method='ffill', limit=1)

c) Reindex again with the same business-day index using `method='ffill'` and `tolerance='1D'`. Explain the difference in output compared to part (b).

```
# Reindex again with the same business-day index using method='ffill' and tolerance='1D'.
# Explain the difference in output compared to part (b)
df_trading = df_trading.reindex(business_days, method='ffill', tolerance='1D')
print(df_trading)

print("\nThe difference here is that tolerance restricts forward-filling at most 1 day\ndifference. Anything after will not be filled in in compar
```

```

AAPL_Price  MSFT_Price  Volume_Million
2023-01-02    150.0      240.0           5.1
2023-01-03    150.0      240.0           0.0
2023-01-04    152.5       NaN           5.3
2023-01-05    152.5       NaN           0.0
2023-01-06     NaN      245.0           5.0
2023-01-09     NaN      245.0           0.0
2023-01-10    155.0      250.0           0.0
2023-01-11    155.0      250.0           0.0
2023-01-12     NaN       NaN           0.0
2023-01-13    157.5       NaN           5.6

```

The difference here is that tolerance restricts forward-filling at most 1 day apart. Anything after will not be filled in in comparison to b)

d) Create a new column "AAPL_Return" representing the daily percentage change of AAPL price (after filling)

```
# Create a new column "AAPL_Return" representing the daily percentage change of AAPL price (after filling)
```

```
df_trading['AAPL_Return'] = df_trading['AAPL_Price'].pct_change() * 100
print(df_trading)
```

```

AAPL_Price  MSFT_Price  Volume_Million  AAPL_Return
2023-01-02    150.0      240.0           5.1         NaN
2023-01-03    150.0      240.0           0.0      0.000000
2023-01-04    152.5       NaN           5.3      1.666667
2023-01-05    152.5       NaN           0.0      0.000000
2023-01-06     NaN      245.0           5.0      0.000000
2023-01-09     NaN      245.0           0.0      0.000000
2023-01-10    155.0      250.0           0.0      1.639344
2023-01-11    155.0      250.0           0.0      0.000000
2023-01-12     NaN       NaN           0.0      0.000000
2023-01-13    157.5       NaN           5.6      1.612903
<ipython-input-161-e2a2360ac1f4>:3: FutureWarning: The default fill_method='pad' in Series.pct_change is deprecated and will be removed
df_trading['AAPL_Return'] = df_trading['AAPL_Price'].pct_change() * 100

```

e) Apply interpolate(method='time') on the price columns and compare the interpolated results with the forward-filled ones using a line plot.

Generated code may be subject to a license | www.analyticsvidhya.com/blog/2022/02/auto-sklearn-accelerate-your-machine-learning-models-with-automl/ | www.analyticsvidhya.com/blog/2021/04/dimens

```

data = {
    "Date": ["2023-01-02", "2023-01-04", "2023-01-06", "2023-01-10", "2023-01-13"],
    "AAPL_Price": [150.0, 152.5, None, 155.0, 157.5],
    "MSFT_Price": [240.0, None, 245.0, 250.0, None],
    "Volume_Million": [5.1, 5.3, 5.0, None, 5.6]
}
df = pd.DataFrame(data)
df["Date"] = pd.to_datetime(df["Date"])
df.set_index("Date", inplace=True)

business_days = pd.date_range(start="2023-01-02", end="2023-01-13", freq="B")
df = df.reindex(business_days)

df_ffill = df.copy()
df_ffill[['AAPL_Price', 'MSFT_Price']] = df_ffill[['AAPL_Price', 'MSFT_Price']].ffill(limit=1)

df_interpolated = df.copy()
df_interpolated[['AAPL_Price', 'MSFT_Price']] = df_interpolated[['AAPL_Price', 'MSFT_Price']].interpolate(method='time')

plt.figure(figsize=(10, 6))

plt.plot(df_ffill.index, df_ffill['AAPL_Price'], label='AAPL - Forward Fill', linestyle='dashed', color='blue')
plt.plot(df_interpolated.index, df_interpolated['AAPL_Price'], label='AAPL - Interpolated', color='red')

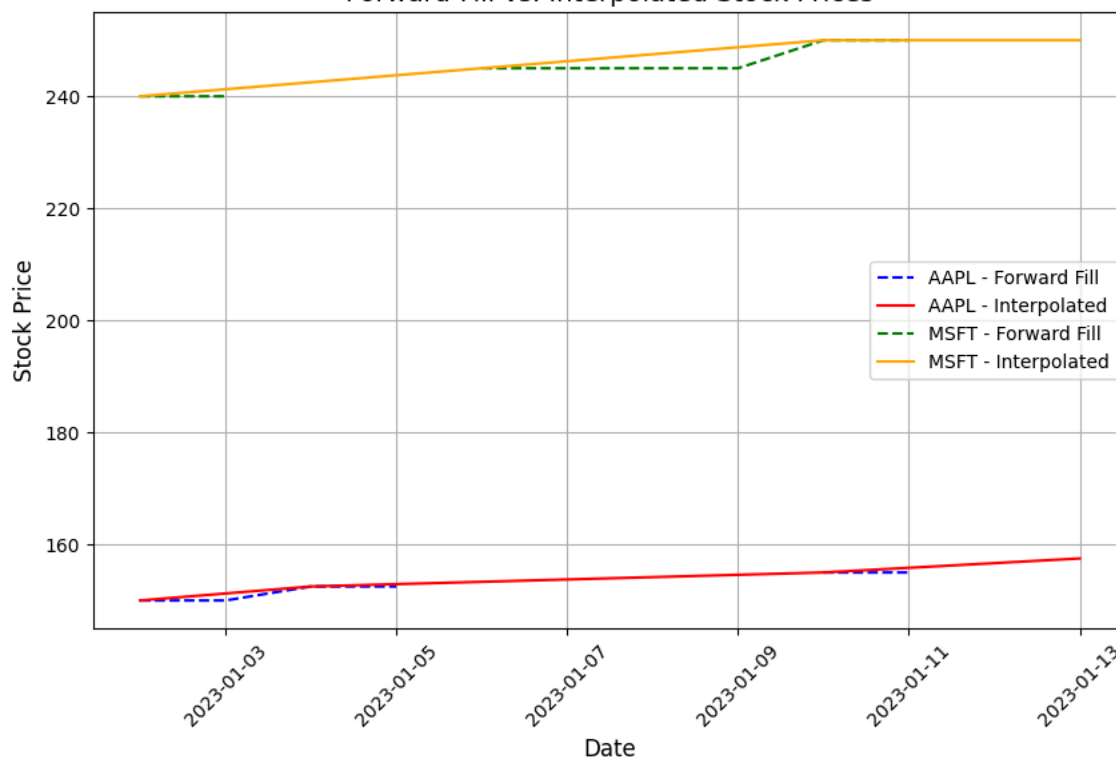
plt.plot(df_ffill.index, df_ffill['MSFT_Price'], label='MSFT - Forward Fill', linestyle='dashed', color='green')
plt.plot(df_interpolated.index, df_interpolated['MSFT_Price'], label='MSFT - Interpolated', color='orange')

plt.xlabel("Date", fontsize=12)
plt.ylabel("Stock Price", fontsize=12)
plt.title("Forward-Fill vs. Interpolated Stock Prices", fontsize=14)
plt.legend()
plt.xticks(rotation=45)
plt.grid(True)
plt.show()

```



Forward-Fill vs. Interpolated Stock Prices



f) Reindex the DataFrame again with the same business-day index using both `copy=True` and `copy=False`. • Compare object identity using `is` or `id()` to show whether memory is shared or duplicated. • Explain when it is preferable to use `copy=False` in performance-critical applications.

```
# Reindex the DataFrame again with the same business-day index using both copy=True and copy=False.
```

```
df_trading_copy_true = df_trading.reindex(business_days, copy=True)
df_trading_copy_false = df_trading.reindex(business_days, copy=False)
print("With copy=True:")
print(df_trading_copy_true)
print("\nWith copy=False:")
print(df_trading_copy_false)
```

```
# Compare object identity using is or id() to show whether memory is shared or duplicated.
```

```
print("Object identity with copy=True:", id(df_trading_copy_true) == id(df_trading))
print("Object identity with copy=False:", id(df_trading_copy_false) == id(df_trading))
```

```
# Explain when it is preferable to use copy=False in performance-critical applications.
```

```
print("\nIt is preferable to use copy = False when the original DataFrame will not be modified after the reindexing operation.\nThis can save memory.")
```



With copy=True:

	AAPL_Price	MSFT_Price	Volume_Million	AAPL_Return
2023-01-02	150.0	240.0	5.1	NaN
2023-01-03	150.0	240.0	0.0	0.000000
2023-01-04	152.5	NaN	5.3	1.666667
2023-01-05	152.5	NaN	0.0	0.000000
2023-01-06	NaN	245.0	5.0	0.000000
2023-01-09	NaN	245.0	0.0	0.000000
2023-01-10	155.0	250.0	0.0	1.639344
2023-01-11	155.0	250.0	0.0	0.000000
2023-01-12	NaN	NaN	0.0	0.000000
2023-01-13	157.5	NaN	5.6	1.612903

With copy=False:

	AAPL_Price	MSFT_Price	Volume_Million	AAPL_Return
2023-01-02	150.0	240.0	5.1	NaN

2023-01-03	150.0	240.0	0.0	0.000000
2023-01-04	152.5	NaN	5.3	1.666667
2023-01-05	152.5	NaN	0.0	0.000000
2023-01-06	NaN	245.0	5.0	0.000000
2023-01-09	NaN	245.0	0.0	0.000000
2023-01-10	155.0	250.0	0.0	1.639344
2023-01-11	155.0	250.0	0.0	0.000000
2023-01-12	NaN	NaN	0.0	0.000000
2023-01-13	157.5	NaN	5.6	1.612903

Object identity with copy=True: False

Object identity with copy=False: False

It is preferable to use `copy = False` when the original DataFrame will not be modified after the reindexing operation. This can save memory and improve performance by avoiding unnecessary copying of data.